The Architecture of Large Language Models

From the Original Transformer to Modern Decoder-Only Models

LLM Theory Reference

April 2026

Abstract

This document provides a top-down architectural reference for the Transformer and its evolution into modern large language models. Starting from the original encoder-decoder Transformer [1], we trace the development through GPT-1 [2] to contemporary decoder-only architectures like LLaMA [4]. Each component is presented with full mathematical formulations, matrix dimensions, and implementation context. The goal is understanding *what* each component does, *why* it exists, and *how* modern models have refined it.

Contents

1 The Original Transformer **3**
- 1.1 Why Attention Replaced Recurrence . . . 3
- 1.2 High-Level Architecture . . . 3

2 Tokenization **3**
- 2.1 The BPE Algorithm . . . 3
- 2.2 Vocabulary Sizes in Practice . . . 4

3 Embeddings **4**
- 3.1 Weight Tying . . . 5
- 3.2 Embedding Scaling . . . 5

4 Positional Encoding **5**
- 4.1 Sinusoidal Positional Encoding (2017) . . . 5
- 4.2 Learned Positional Embeddings (2018) . . . 6
- 4.3 Rotary Position Embedding — RoPE (2021) . . . 6

5 Attention Mechanism **6**
- 5.1 Scaled Dot-Product Attention . . . 7
- 5.2 Multi-Head Attention . . . 7
- 5.3 Three Types of Attention . . . 8

6 Feed-Forward Network **8**
- 6.1 Original FFN (ReLU) . . . 8
- 6.2 GELU Activation (GPT-1) . . . 8
- 6.3 SwiGLU (LLaMA and Modern Models) . . . 9

7 Normalization and Residual Connections **9**
7.1 Residual Connections . . . 9
7.2 Layer Normalization . . . 10
7.3 Post-LN vs. Pre-LN . . . 10
7.4 RMSNorm . . . 10

8 The Decoder-Only Shift **11**
8.1 From Encoder-Decoder to Decoder-Only . . . 11
8.2 Causal Masking . . . 11
8.3 Why Decoder-Only Won . . . 11
8.4 Architectural Timeline . . . 12

9 Output Head **12**
9.1 Final Layer Normalization . . . 12
9.2 Projection to Logits . . . 12
9.3 Softmax . . . 12

10 Putting It Together **13**
10.1 Step-by-Step Forward Pass . . . 13
10.2 Parameter Count Breakdown . . . 14

1 The Original Transformer

The Transformer [1] was introduced in 2017 as a sequence-to-sequence model for machine translation. Its key innovation was replacing recurrence entirely with *attention mechanisms*, allowing the model to relate any two positions in a sequence in constant time.

1.1 Why Attention Replaced Recurrence

Prior architectures (RNNs, LSTMs) processed sequences one step at a time, creating a fundamental bottleneck: information from early tokens had to survive through every intermediate step to reach later tokens. The Transformer eliminates this by letting every position attend directly to every other position.

The computational trade-offs per layer are [1]:

Layer Type	Complexity per Layer	Sequential Operations
Self-Attention	$O(n^2 \cdot d)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$

where n is the sequence length and d is the representation dimension. Self-attention is more expensive per layer when $n > d$, but its $O(1)$ sequential operations (fully parallelizable) make it dramatically faster on modern hardware. For typical LLM configurations ($d = 4096$, $n \leq 8192$), both terms are comparable, and the parallelism advantage dominates.

1.2 High-Level Architecture

The original Transformer is an *encoder-decoder* model:

- **Encoder** (left): Processes the input sequence. Consists of N identical layers, each containing a self-attention sub-layer followed by a feed-forward network. The encoder sees the entire input at once (bidirectional).
- **Decoder** (right): Generates the output sequence one token at a time. Also N layers, but each layer has three sub-layers: *masked* self-attention (can only attend to earlier positions), encoder-decoder cross-attention (attends to encoder output), and a feed-forward network.

Both stacks use residual connections and layer normalization around each sub-layer. The base model uses $N = 6$ layers in each stack, with $d_{\text{model}} = 512$ [1].

Concrete Dimensions

Original Transformer (base): $N = 6$ layers, $d_{\text{model}} = 512$, $h = 8$ heads, $d_k = d_v = d_{\text{model}}/h = 64$, $d_{\text{ff}} = 2048$. Total parameters: 65M.

2 Tokenization

Before a model can process text, it must convert characters into discrete integer tokens. The dominant method is **Byte Pair Encoding** (BPE) [5], adapted from a data compression algorithm.

2.1 The BPE Algorithm

BPE builds a vocabulary of subword units by iteratively merging the most frequent adjacent pairs:

Algorithm: BPE vocabulary construction [5]

1. Initialize vocabulary $\mathcal{V}$ with all individual characters (or bytes)
2. Represent each word in the training corpus as a sequence of characters
3. **For** $i = 1$ to k (desired number of merges):
   a. Count all adjacent symbol pairs across the corpus
   b. Find the most frequent pair (a, b)
   c. Merge every occurrence of (a, b) into a new symbol ab
   d. Add ab to vocabulary $\mathcal{V}$
4. **Return** $\mathcal{V}$

The only hyperparameter is the number of merge operations k, which directly controls the final vocabulary size: $|\mathcal{V}| = |\text{base characters}| + k$.

Example. Starting from the corpus {"low", "lowest", "newer", "wider"}, BPE might learn merges: `l o` $\rightarrow$ `lo`, then `lo w` $\rightarrow$ `low`, then `e r` $\rightarrow$ `er`. After these merges, the unseen word "lower" segments as `low er` — two known tokens [5].

2.2 Vocabulary Sizes in Practice

Model	Vocab Size	Tokenizer
Original Transformer [1]	37,000	BPE (shared source/target)
GPT-1 [2]	40,000	BPE
GPT-3 [3]	50,257	BPE
LLaMA [4]	32,000	SentencePiece BPE (byte-fallback)

Implementation Note

LLaMA uses SentencePiece [4], which operates on raw byte sequences rather than Unicode characters. This provides a *byte-fallback* mechanism: any byte sequence can be tokenized, even if it contains characters unseen during training. All numbers are split into individual digits. This means the vocabulary is truly closed — there are no "unknown token" failures at inference time.

3 Embeddings

Each token index $t \in \{0, 1, \ldots, V-1\}$ is mapped to a dense vector via the **token embedding matrix**:

$$W_e \in \mathbb{R}^{V \times d_{\text{model}}} \tag{1}$$

where V is the vocabulary size and d_{model} is the model dimension. Looking up token t simply selects the t-th row of W_e.

For a sequence of tokens $(t_1, t_2, \ldots, t_n)$, the embedding step produces a matrix:

$$X_{\text{embed}} = \begin{pmatrix} W_e[t_1] \\ W_e[t_2] \\ \vdots \\ W_e[t_n] \end{pmatrix} \in \mathbb{R}^{n \times d_{\text{model}}} \tag{2}$$

3.1 Weight Tying

A key architectural detail: in both the original Transformer [1] and GPT-1 [2], the *same* weight matrix W_e is used for three purposes:

1. Source (encoder) input embeddings
2. Target (decoder) input embeddings
3. The output projection that converts hidden states back to vocabulary logits (see Section 9)

In decoder-only models (which have no encoder), points 1 and 2 collapse, so W_e is shared between the input embedding and the output projection:

$$\text{logits} = h_n \cdot W_e^\top \tag{3}$$

where $h_n \in \mathbb{R}^{d_{\text{model}}}$ is the final hidden state. This halves the parameters devoted to vocabulary (which can be substantial — for LLaMA-7B with $V = 32000$ and $d_{\text{model}} = 4096$, the embedding matrix alone is $32000 \times 4096 \approx 131\text{M}$ parameters).

3.2 Embedding Scaling

The original Transformer scales embeddings by $\sqrt{d_{\text{model}}}$ before adding positional encodings [1]:

$$X = \sqrt{d_{\text{model}}} \cdot X_{\text{embed}} + X_{\text{pos}} \tag{4}$$

where $X_{\text{pos}} \in \mathbb{R}^{n \times d_{\text{model}}}$ is the positional encoding matrix (Section 4). This compensates for the fact that embedding vectors tend to have small magnitudes (roughly unit variance per dimension), while positional encodings have magnitudes on the order of 1. The scaling ensures the token identity signal is not overwhelmed by positional information. GPT-1 and LLaMA do not use this scaling factor.

4 Positional Encoding

Self-attention is *permutation-invariant*: if you shuffle the input tokens, the attention computation produces the same outputs (just shuffled). The model has no inherent sense of token order. Positional encodings inject sequence-position information so the model can distinguish "the cat sat on the mat" from "mat the on sat cat the."

Three major approaches have been used, representing an evolution in how position is encoded.

4.1 Sinusoidal Positional Encoding (2017)

The original Transformer [1] adds fixed sinusoidal functions to the embeddings. For position pos and dimension i:

$$PE_{(\text{pos},2i)} = \sin\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big) \tag{5}$$

$$PE_{(\text{pos},2i+1)} = \cos\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big) \tag{6}$$

Each dimension oscillates at a different frequency, from 2π (dimension 0) to $10000 \cdot 2\pi$ (last dimension). The intuition: position is encoded as a binary-like code where each dimension captures a different "bit" of the position at a different time scale.

A key property: for any fixed offset k, $PE_{\text{pos}+k}$ can be expressed as a linear function of PE_{pos}. This means the model can learn to attend to relative positions using linear projections.

These encodings are *added* to the token embeddings (not concatenated), so they share the same d_{model}-dimensional space.

4.2 Learned Positional Embeddings (2018)

GPT-1 [2] replaced the fixed sinusoidal functions with a *learned* position embedding matrix:

$$W_p \in \mathbb{R}^{n_{\max} \times d_{\text{model}}} \tag{7}$$

where $n_{\max}$ is the maximum sequence length (512 for GPT-1). The position embedding for position pos is simply the pos-th row of W_p, trained jointly with the rest of the model:

$$h_0 = X_{\text{embed}} \cdot W_e + W_p \tag{8}$$

This gives the model full freedom to learn whatever positional representation is most useful, but fixes the maximum context length at training time.

4.3 Rotary Position Embedding — RoPE (2021)

RoPE [6] takes a fundamentally different approach: instead of *adding* position to the embeddings, it *rotates* the query and key vectors by position-dependent angles. This makes the dot product between query at position m and key at position n depend only on their relative distance $(m-n)$.

For a vector $\boldsymbol{x} \in \mathbb{R}^d$, RoPE groups dimensions into pairs and applies a 2D rotation to each pair. The rotation angle for pair i at position m is:

$$\theta_i = m \cdot \omega_i, \quad \text{where } \omega_i = 10000^{-2i/d} \tag{9}$$

where m is the token position in the sequence, $i \in \{0, 1, \ldots, d/2-1\}$ is the dimension pair index, and d is the head dimension (d_k).

The rotation is applied as:

$$f(\boldsymbol{x}, m) = \begin{pmatrix} x_0 \cos(m\omega_0) - x_1 \sin(m\omega_0) \\ x_0 \sin(m\omega_0) + x_1 \cos(m\omega_0) \\ x_2 \cos(m\omega_1) - x_3 \sin(m\omega_1) \\ x_2 \sin(m\omega_1) + x_3 \cos(m\omega_1) \\ \vdots \end{pmatrix} \tag{10}$$

where $x_0, x_1, \ldots$ are the scalar components of $\boldsymbol{x}$.

The key mathematical property is that for rotated queries $\boldsymbol{q}_m = f(\boldsymbol{q}, m)$ and keys $\boldsymbol{k}_n = f(\boldsymbol{k}, n)$:

$$\boldsymbol{q}_m^\top \boldsymbol{k}_n = \boldsymbol{q}^\top R_{m-n} \boldsymbol{k} \tag{11}$$

where R_{m-n} is a rotation matrix depending only on the relative position $(m-n)$. The dot product — and thus the attention weight — naturally encodes relative position without any explicit relative position bias.

Architectural Evolution

RoPE is applied only to the query Q and key K vectors, *not* to values V [6]. It is applied at every layer, not just the first. This is the positional encoding used by LLaMA [4] and most modern LLMs.

5 Attention Mechanism

Attention is the core operation of the Transformer. Conceptually, it allows each position in a sequence to "look at" every other position and aggregate information based on relevance.

5.1 Scaled Dot-Product Attention

Given queries Q, keys K, and values V, attention computes [1]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{12}$$

where $Q, K \in \mathbb{R}^{n \times d_k}$ are the query and key matrices, $V \in \mathbb{R}^{n \times d_v}$ is the value matrix, n is the sequence length, and d_k is the key/query dimension (the scaling factor).

Step by step, for a single query vector $\boldsymbol{q}$ attending to keys K and values V:

1. **Compute similarity scores:** $\boldsymbol{s} = \boldsymbol{q}K^\top \in \mathbb{R}^n$. Each score $s_j = \boldsymbol{q} \cdot \boldsymbol{k}_j$ measures how relevant position j is.
2. **Scale:** $\boldsymbol{s} \leftarrow \boldsymbol{s}/\sqrt{d_k}$. The scaling prevents the dot products from growing large in magnitude with d_k, which would push the softmax into regions of extremely small gradients. Without scaling, for random unit-variance vectors, $\text{Var}(\boldsymbol{q} \cdot \boldsymbol{k}) = d_k$, so dividing by $\sqrt{d_k}$ restores unit variance [1].
3. **Softmax:** $\boldsymbol{a} = \text{softmax}(\boldsymbol{s}) \in \mathbb{R}^n$. Converts scores to a probability distribution (attention weights). Each $a_j \geq 0$ and $\sum_j a_j = 1$.
4. **Weighted sum:** output $= \sum_j a_j \boldsymbol{v}_j$. The output is a weighted combination of value vectors, where the weights are the attention scores.

In matrix form for all n positions simultaneously:

$$Q, K, V \in \mathbb{R}^{n \times d_k}, \quad QK^\top \in \mathbb{R}^{n \times n}, \quad \text{Attention}(Q, K, V) \in \mathbb{R}^{n \times d_v} \tag{13}$$

The $n \times n$ attention matrix is the computational bottleneck — it's why self-attention is $O(n^2)$ in sequence length.

5.2 Multi-Head Attention

Rather than performing a single attention function with d_{model}-dimensional keys, values, and queries, multi-head attention runs h attention operations in parallel, each on a different learned linear projection [1]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \tag{14}$$

$$\text{where head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V) \tag{15}$$

The projection matrices are:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \quad W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}} \tag{16}$$

With $d_k = d_v = d_{\text{model}}/h$, the total computation is similar to single-head attention with full dimensionality, but the model can attend to information from different representation subspaces at different positions simultaneously.

Concrete Dimensions

Original Transformer: $d_{\text{model}} = 512$, $h = 8$, so $d_k = d_v = 64$. Each head operates on 64-dimensional projections.

LLaMA-7B: $d_{\text{model}} = 4096$, $h = 32$, so $d_k = d_v = 128$. With RoPE applied to Q and K at every layer.

5.3 Three Types of Attention

The original encoder-decoder Transformer uses three distinct attention patterns [1]:

1. **Encoder self-attention:** Q, K, V all come from the encoder. Every position can attend to every other position (bidirectional). The attention matrix is dense.
2. **Decoder masked self-attention:** Q, K, V all come from the decoder. Position i can only attend to positions $\leq i$ (autoregressive / causal). This is enforced by setting the upper-triangular entries of $QK^\top$ to $-\infty$ before the softmax, so they become zero:

$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{17}$$

   where i is the query position and j is the key position.
3. **Encoder-decoder cross-attention:** Q comes from the decoder, K and V come from the encoder output. This is how the decoder "reads" the input. Each decoder position can attend to all encoder positions.

In decoder-only models (GPT, LLaMA), only type 2 exists. Cross-attention and bidirectional self-attention are removed entirely (Section 8).

6 Feed-Forward Network

Each Transformer layer contains a **position-wise feed-forward network** (FFN) — a two-layer fully-connected network applied independently and identically to each position in the sequence [1]. While attention mixes information *across* positions, the FFN transforms the representation *at* each position.

6.1 Original FFN (ReLU)

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \tag{18}$$

where $x \in \mathbb{R}^{d_{\text{model}}}$ is the hidden state at a single position, $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ are weight matrices, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$ are bias vectors, and $\text{ReLU}(z) = \max(0, z)$.

The inner dimension d_{ff} is typically $4 \times d_{\text{model}}$. In the original Transformer, $d_{\text{model}} = 512$ and $d_{\text{ff}} = 2048$, so the FFN "expands" the representation to 4x width, applies a nonlinearity, then "compresses" back. This expansion is thought to provide capacity for the model to process and transform features.

6.2 GELU Activation (GPT-1)

GPT-1 [2] replaced ReLU with the **Gaussian Error Linear Unit** (GELU):

$$\text{GELU}(x) = x \cdot \Phi(x) \tag{19}$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. Unlike ReLU, which hard-clips at zero, GELU applies a smooth, input-dependent gate. Values near zero are partially attenuated rather than fully zeroed.

$$\text{FFN}_{\text{GELU}}(x) = \text{GELU}(xW_1)W_2 \tag{20}$$

where x, W_1, and W_2 are as defined in Equation (18) (without bias terms).

(GPT-1 also drops the bias terms, following the T5 convention [7].)

6.3 SwiGLU (LLaMA and Modern Models)

Shazeer [7] introduced Gated Linear Unit (GLU) variants into the Transformer FFN. The key idea is to use a *gating mechanism*: one linear projection produces the content, another produces a gate that controls what passes through.

SwiGLU combines the Swish activation function with a gated linear unit:

$$\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)\, W_2 \tag{21}$$

where $x \in \mathbb{R}^{d_{\text{model}}}$ is the input hidden state, $W, V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are the gate and up-projection weight matrices, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the down-projection weight matrix, $\otimes$ is element-wise multiplication, $\text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$, and σ is the sigmoid function. When $\beta = 1$, this is also called SiLU.

GLU variants use *three* weight matrices (W, V, W_2) instead of the standard two (W_1, W_2). To keep the parameter count and computation equivalent, the inner dimension d_{ff} is reduced by a factor of $\frac{2}{3}$ [7]:

$$d_{\text{ff}}^{\text{SwiGLU}} = \frac{2}{3} \cdot 4 d_{\text{model}} \tag{22}$$

Concrete Dimensions

LLaMA-7B: $d_{\text{model}} = 4096$, $d_{\text{ff}} = \frac{2}{3} \times 4 \times 4096 = 10922 \approx 11008$ (rounded to a multiple of 256 for hardware efficiency) [4]. Compare to the standard $4 \times 4096 = 16384$ that a non-gated FFN would use.

Architectural Evolution

FFN activation evolution:

- 2017: ReLU [1] — hard zero below 0
- 2018: GELU [2] — smooth, probabilistic gating
- 2020: SwiGLU [7] — gated architecture with Swish activation

SwiGLU consistently achieves better perplexity than ReLU and GELU in matched-parameter comparisons [7], and is used in LLaMA [4] and PaLM. The original paper offers "no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence" [7].

7 Normalization and Residual Connections

Every sub-layer (attention, FFN) in the Transformer is wrapped in a residual connection and a normalization step. The placement of normalization relative to the sub-layer has significant implications for training dynamics.

7.1 Residual Connections

Each sub-layer output is added to its input via a skip connection:

$$\text{output} = x + \text{SubLayer}(x) \tag{23}$$

where x is the sub-layer input and $\text{SubLayer}(\cdot)$ is either the attention or FFN function.

Residual connections serve two critical purposes:

1. **Gradient flow:** They create a direct path for gradients to flow from the loss back to early layers, mitigating the vanishing gradient problem in deep networks.
2. **Incremental refinement:** Each layer's job becomes learning a *correction* to its input rather than computing the entire representation from scratch. This makes optimization easier.

7.2 Layer Normalization

Layer normalization [8] normalizes across the feature dimension for each individual position and sample:

$$\text{LayerNorm}(\boldsymbol{v}) = \gamma \cdot \frac{\boldsymbol{v} - \mu}{\sigma} + \beta \tag{24}$$

where $\boldsymbol{v} \in \mathbb{R}^d$ is the input vector (d is the feature dimension, i.e. d_{model}), $\mu = \frac{1}{d}\sum_{k=1}^{d} v_k$ is the mean, $\sigma = \sqrt{\frac{1}{d}\sum_{k=1}^{d}(v_k - \mu)^2}$ is the standard deviation, and $\gamma, \beta \in \mathbb{R}^d$ are learned scale and bias parameters.

7.3 Post-LN vs. Pre-LN

The placement of layer normalization has major implications for training stability [8]:

Post-LN (original Transformer [1]):

$$x' = \text{LayerNorm}(x + \text{SubLayer}(x)) \tag{25}$$

where x is the layer input and x' is the layer output. The sub-layer output is added to the input first, *then* normalized.

Pre-LN (GPT and modern models):

$$x' = x + \text{SubLayer}(\text{LayerNorm}(x)) \tag{26}$$

The input is normalized first, *then* passed through the sub-layer, then added back. Pre-LN also requires a *final* layer normalization before the output projection.

Xiong et al. [8] proved via mean field theory that these placements have different gradient behaviors at initialization:

Variant	Output-layer gradient scale	Warmup needed?
Post-LN	$O(d\sqrt{\ln d})$, independent of L	Yes (critical)
Pre-LN	$O\left(d\sqrt{\frac{\ln d}{L}}\right)$, decreases with depth	No

where d is the model dimension (d_{model}) and L is the number of layers.

In Post-LN, gradients near the output layer are large regardless of model depth, making training unstable without careful learning rate warmup. In Pre-LN, gradients scale inversely with $\sqrt{L}$ (number of layers), keeping them well-behaved even for deep models. This is why Pre-LN enables training without warmup and is used by virtually all modern LLMs.

7.4 RMSNorm

RMSNorm [9] simplifies layer normalization by removing the mean-centering step:

$$\text{RMSNorm}(\boldsymbol{v}) = \gamma \cdot \frac{\boldsymbol{v}}{\text{RMS}(\boldsymbol{v})}, \quad \text{where RMS}(\boldsymbol{v}) = \sqrt{\frac{1}{d}\sum_{k=1}^{d} v_k^2} \tag{27}$$

where $\boldsymbol{v} \in \mathbb{R}^d$ is the input vector, d is the feature dimension, v_k are the scalar components of $\boldsymbol{v}$, and $\gamma \in \mathbb{R}^d$ is a learned scale parameter.

By dropping the mean subtraction and the bias term β, RMSNorm provides:

- Comparable model quality to standard LayerNorm
- 7–64% reduction in computation time [9]

LLaMA [4] uses Pre-LN with RMSNorm — this is now the standard configuration for modern LLMs.

Architectural Evolution

Normalization evolution:

- 2017: Post-LN with standard LayerNorm [1] — requires warmup
- 2018: Pre-LN with standard LayerNorm [2] — no warmup needed
- 2023: Pre-LN with RMSNorm [4] — no warmup, faster computation

8 The Decoder-Only Shift

Modern LLMs (GPT, LLaMA, Claude, Mistral, etc.) are all *decoder-only* Transformers. This section traces why and how the field converged on this architecture.

8.1 From Encoder-Decoder to Decoder-Only

GPT-1 [2] made the critical architectural choice: take the Transformer decoder, remove the encoder-decoder cross-attention sub-layer, and use the resulting model for language modeling. The architecture reduces to:

$$h_0 = X_{\text{embed}} \cdot W_e + W_p \tag{28}$$

$$h_l = \text{transformer\_block}(h_{l-1}) \quad \forall l \in [1, L] \tag{29}$$

$$P(t) = \text{softmax}(h_L \cdot W_e^\top) \tag{30}$$

where $h_0 \in \mathbb{R}^{n \times d_{\text{model}}}$ is the initial hidden state, X_{embed} is the token index matrix, W_e is the embedding matrix, $W_p \in \mathbb{R}^{n_{\max} \times d_{\text{model}}}$ is the learned position embedding, h_l is the hidden state after layer l, L is the total number of layers, and $P(t) \in \mathbb{R}^V$ is the output probability distribution over the vocabulary.

Each transformer block consists of just two sub-layers:

1. Masked multi-head self-attention
2. Position-wise feed-forward network

8.2 Causal Masking

The defining feature of decoder-only models is the **causal mask**: token at position i can only attend to tokens at positions $\leq i$. This is implemented by adding a mask to the attention scores before the softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \tag{31}$$

where $M \in \mathbb{R}^{n \times n}$ has $M_{ij} = 0$ if $j \leq i$ and $M_{ij} = -\infty$ if $j > i$.

This enforces autoregressive generation: the prediction for position i depends only on tokens $1, \ldots, i$, matching the language modeling objective $P(t_i \mid t_1, \ldots, t_{i-1})$.

8.3 Why Decoder-Only Won

The encoder-decoder structure was designed for sequence-to-sequence tasks (translation), where you have a complete input and generate a complete output. For general language modeling and generation:

- There is no separate "input" to encode — the model simply continues a sequence.

- The decoder alone is sufficient for the autoregressive objective.
- Removing the encoder and cross-attention cuts the architecture roughly in half, simplifying both the model and the training pipeline.
- The same model serves both understanding and generation — no separate encoder needed.

8.4 Architectural Timeline

Model	Year	Key changes from Transformer
Transformer [1]	2017	Encoder-decoder, Post-LN, sinusoidal PE, ReLU FFN
GPT-1 [2]	2018	Decoder-only, Pre-LN[1], learned PE, GELU, BPE 40K
GPT-3 [3]	2020	Scaled to 175B params (96 layers, d=12288, 96 heads). Alternating dense and locally-banded sparse attention in some layers
LLaMA [4]	2023	Pre-RMSNorm, SwiGLU ($d_{\text{ff}} = \frac{2}{3} \cdot 4d$), RoPE at every layer, no bias terms, SentencePiece BPE. Sizes 7B–65B, trained on public data only

9 Output Head

The output head converts the final hidden state into a probability distribution over the vocabulary.

9.1 Final Layer Normalization

In Pre-LN models, the residual stream after the last layer has not been normalized (since Pre-LN normalizes *before* each sub-layer). A final layer normalization is therefore applied before the output projection [8]:

$$h_{\text{final}} = \text{RMSNorm}(h_L) \quad \text{(or LayerNorm in older models)} \tag{32}$$

where $h_L \in \mathbb{R}^{n \times d_{\text{model}}}$ is the output of the last Transformer layer (L layers total).

9.2 Projection to Logits

The normalized hidden state is projected to vocabulary-sized logits:

$$\boldsymbol{z} = h_{\text{final}} \cdot W_e^{\top} \in \mathbb{R}^V \tag{33}$$

where $W_e \in \mathbb{R}^{V \times d_{\text{model}}}$ is the (shared) embedding matrix (Section 3.1). Each element z_j is the unnormalized log-probability (logit) for vocabulary token j.

9.3 Softmax

The logits are converted to probabilities:

$$P(t = j \mid t_1, \ldots, t_{i-1}) = \frac{e^{z_j}}{\sum_{k=0}^{V-1} e^{z_k}} \tag{34}$$

[1] GPT-1's Figure 1 shows Layer Norm placed before each sub-layer (the Pre-LN arrangement), which differs from the original Transformer's Post-LN.

where z_j is the logit for token j, and V is the vocabulary size.

During training, the target is the next token t_i, and the loss is the cross-entropy:

$$\mathcal{L} = -\log P(t_i \mid t_1, \ldots, t_{i-1}) \tag{35}$$

summed over all positions in the sequence.

During inference, a token is selected from this distribution (via greedy decoding, top-k sampling, nucleus sampling, etc. — these are inference-time strategies outside the scope of this document).

10 Putting It Together

Let us trace a complete forward pass through a modern decoder-only Transformer, using LLaMA-7B dimensions [4] as a concrete example.

Concrete Dimensions

LLaMA-7B hyperparameters: $d_{\text{model}} = 4096$, $L = 32$ layers, $h = 32$ heads, $d_k = d_v = 128$, $d_{\text{ff}} = 11008$, $V = 32{,}000$. Total: $\sim$6.7B parameters.

10.1 Step-by-Step Forward Pass

Given an input sequence of n token indices $(t_1, \ldots, t_n)$:

Step 1: Token Embedding

$$X = W_e[t_1, \ldots, t_n] \in \mathbb{R}^{n \times 4096} \tag{36}$$

Look up each token index in $W_e \in \mathbb{R}^{32000 \times 4096}$.

Step 2: Repeat for each of $L = 32$ layers. Let $h^{(0)} = X$.

For layer $l = 1, \ldots, 32$:

Step 2a: Pre-RMSNorm (attention)

$$\hat{h} = \text{RMSNorm}(h^{(l-1)}) \in \mathbb{R}^{n \times 4096} \tag{37}$$

Step 2b: Multi-head self-attention with RoPE

$$Q = \hat{h}\, W^Q \in \mathbb{R}^{n \times 4096} \quad \text{(reshaped to } n \times 32 \times 128\text{)} \tag{38}$$

$$K = \hat{h}\, W^K \in \mathbb{R}^{n \times 4096} \quad \text{(reshaped to } n \times 32 \times 128\text{)} \tag{39}$$

$$V = \hat{h}\, W^V \in \mathbb{R}^{n \times 4096} \quad \text{(reshaped to } n \times 32 \times 128\text{)} \tag{40}$$

Apply RoPE rotations to Q and K based on position.

For each head i, compute:

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{128}} + M\right) V_i \in \mathbb{R}^{n \times 128} \tag{41}$$

where M is the causal mask. Concatenate all 32 heads and project:

$$A = \text{Concat}(\text{head}_1, \ldots, \text{head}_{32})\, W^O \in \mathbb{R}^{n \times 4096} \tag{42}$$

Step 2c: Residual connection (attention)

$$h' = h^{(l-1)} + A \tag{43}$$

Step 2d: Pre-RMSNorm (FFN)

$$\hat{h}' = \text{RMSNorm}(h') \in \mathbb{R}^{n \times 4096} \tag{44}$$

Step 2e: SwiGLU FFN

$$F = \left(\text{Swish}(\hat{h}' W_{\text{gate}}) \otimes \hat{h}' W_{\text{up}}\right) W_{\text{down}} \in \mathbb{R}^{n \times 4096} \tag{45}$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{4096 \times 11008}$ and $W_{\text{down}} \in \mathbb{R}^{11008 \times 4096}$.

Step 2f: Residual connection (FFN)

$$h^{(l)} = h' + F \tag{46}$$

Step 3: Final RMSNorm

$$h_{\text{final}} = \text{RMSNorm}(h^{(32)}) \in \mathbb{R}^{n \times 4096} \tag{47}$$

Step 4: Output projection

$$\text{logits} = h_{\text{final}} \cdot W_e^{\top} \in \mathbb{R}^{n \times 32000} \tag{48}$$

Step 5: Softmax (per position)

$$P(t_{i+1} \mid t_1, \ldots, t_i) = \text{softmax}(\text{logits}[i]) \in \mathbb{R}^{32000} \tag{49}$$

10.2 Parameter Count Breakdown

Component	Params per layer	$\times$ 32 layers	Total
Attention (W^Q, W^K, W^V, W^O)	4×4096^2		2.1B
FFN ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)	$3 \times 4096 \times 11008$		4.3B
RMSNorm ($\gamma \times 2$)	2×4096		8.4M
All layers			6.4B
Token embedding W_e			131M
Final RMSNorm			4K
Total			**$\sim$6.7B**

Note: the FFN accounts for roughly two-thirds of per-layer parameters, and attention accounts for one-third. This ratio holds across model sizes.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
https://arxiv.org/abs/1706.03762

[2] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018.
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[3] T. B. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
https://arxiv.org/abs/2005.14165

[4] H. Touvron, T. Lavril, G. Izacard, et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.
https://arxiv.org/abs/2302.13971

[5] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," *Proceedings of the 54th Annual Meeting of the ACL*, 2016.
https://arxiv.org/abs/1508.07909

[6] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv preprint arXiv:2104.09864, 2021.
https://arxiv.org/abs/2104.09864

[7] N. Shazeer, "GLU Variants Improve Transformer," arXiv preprint arXiv:2002.05202, 2020.
https://arxiv.org/abs/2002.05202

[8] R. Xiong, Y. Yang, D. He, et al., "On Layer Normalization in the Transformer Architecture," *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
https://arxiv.org/abs/2002.04745

[9] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
https://arxiv.org/abs/1910.07467